Australian National University

COMP4610 / COMP8610 – Semester 1, 2026: **Research Project-1**

Declaration of Originality

I, *Anusha Garg, Samuel Joffy, Charles Kelly*, *Mukund Balaji Srinivas* UID: *u8035584, u7903178, u7485509, u7274095* here declare that the code and report submitted in this assignment are my own work. The programs and results presented in this report were developed and written independently by me.

I confirm that I have **not copied code, text, or results from other students or external sources**, except where properly acknowledged. I understand that plagiarism or any form of academic dishonesty is not permitted.

AI Use: In completing this assignment, I declare that I:

☐ **Have not used any AI tools**

✓ **Have used AI tools** (e.g., ChatGPT, DeepSeek, or similar AI systems)

If you used AI tools, please briefly describe **how they were used**. For example:

- grammar or language editing
- help understanding the tasks and concepts
- debugging suggestions
- AI coding
- other assistance (please specify below)

Description of AI use (if applicable):

Ai was used to assist with structuring and refining the project report. All technical implementation, design decisions and testing were carried out by the team independently.

Student Signature (type your name): ___Anusha, Samuel, Charles, Mukund___

General Instructions

Please include this page as **the first page of your project report** (this page does **not** count toward the page limit).

COMP8610/COMP4610- Computer Graphics
Research Project 1
Team Dolly

Animate a 3D cubism-style robot or animal character

Abstract

This report presents the design and implementation of a three-dimensional, cubism-inspired animated dog developed for COMP8610 Research Project 1. The project integrates hierarchical modelling, procedural animation, and real-time user interaction, implemented using C++ and OpenGL. The walking motion is generated using mathematical wave functions, enabling a smooth and continuous animation cycle. In addition, the system allows real-time control of the character through keyboard input, demonstrating an effective combination of modelling, animation, and interaction techniques.

Introduction

This project addresses Option 1 from the COMP8610 Research Project 1 brief. The primary objective was to construct a cubism-style dog model using simple cuboid primitives, animate it with a walking cycle, and extend it into an interactive application responsive to user input.

The project work was divided among four team members, covering character modelling and animation, rendering, input processing, and report preparation. The system follows a structured pipeline: a model-generation function produces the dog's mesh for a given point in the animation cycle, the input handler updates the character's state based on user interactions, and the renderer displays the updated model on screen for each frame.

Character Structure

The dog model is constructed entirely from cuboid primitives organised within a hierarchical structure. The body serves as the root node, with the tail, head, and four legs branching directly from it. The head further acts as a parent node to additional components, including the snout, left ear, and right ear. In total, the model consists of ten distinct parts and more than six independently controllable joints, satisfying the project requirements.

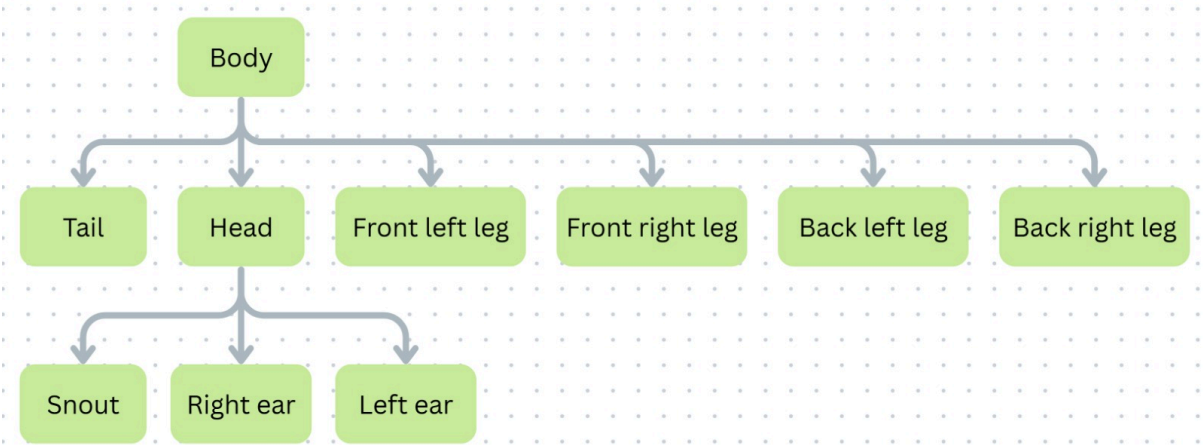


Figure 1: Hierarchy tree of the dog model

This hierarchical design provides a significant advantage during animation, as transformations applied to a parent node automatically propagate to its child components. For example, when the head moves, the snout and ears move correspondingly without requiring separate animation for each part. This simplifies the animation process by allowing each joint to be defined relative to its parent rather than in global coordinate space. Additionally, it enables the incorporation of animation principles, such as the ears reacting naturally to head movement, which is discussed further in the following section.

Animation Method

Rather than storing predefined keyframes, the animation is driven entirely by a single function called `buildAnimalModelAtPoint(float point)`. Given a value in the range [0, 1), the function returns the complete mesh of the dog at that point in the walk cycle. For example, a value of 0 returns the model at the start of the cycle, while a value of 0.999 returns the model immediately before the loop restarts. All motion is computed mathematically at runtime without relying on stored animation data.

The rotation of each body part is determined using sine and cosine wave functions applied to the input value. For the legs, a diagonal pairing strategy is used. The front left and back right legs share a common scalar value $l = \sin(\pi t)$, where $t = 2*\text{point}$, producing a rotation of $25l$ degrees along the x-axis. The remaining pair, consisting of the front right and back left legs, applies a rotation of $-25l$ degrees, ensuring they always move in the opposite direction. This diagonal pairing produces the natural alternating gait characteristic of a walking quadruped.

Over the course of one complete cycle, the motion progresses as follows. At $t = 0$, all legs are in their neutral position pointing straight down. By $t = 0.5$, l reaches its maximum value of 1, causing one diagonal pair to swing fully backward and the other fully forward. At $t = 1$, all legs return to the neutral position. By $t = 1.5$, the diagonal pairs are in reversed positions compared to $t = 0.5$, and at $t = 2$ the cycle completes and begins again.

To reduce the rigidity of the motion, the head and tail are also animated to move vertically in synchronisation with the walking cycle, simulating the natural weight shift of a moving animal. The ears introduce an additional layer of detail by moving on a slight delay relative to the head, rather than in perfect synchronisation. This behaviour is consistent with the follow-through principle of animation, whereby objects connected to a moving body follow the same movement path with a slight temporal delay. Although subtle, this detail contributes meaningfully to the overall naturalism of the animation.

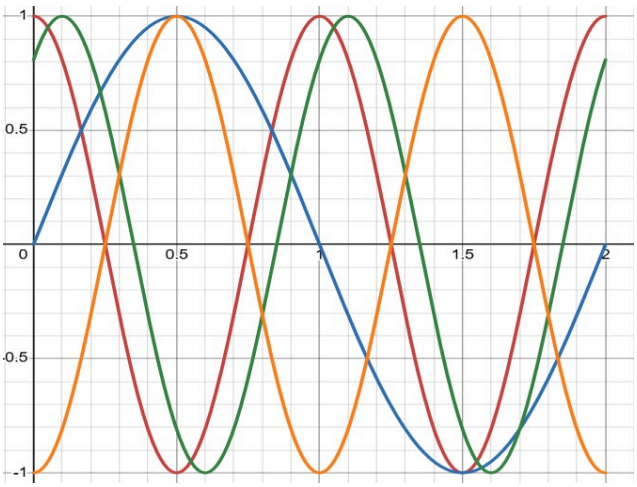


Figure 2: Graph of limb motion scalars over one full cycle. Blue = legs, Red = head, Green = ears, Orange = tail

Control Logic (Task 2)

For Task 2, the animation system was extended into a fully interactive application. At each frame, the `handleInput` function defined in `input.cpp` is invoked to determine which keys are currently being held, and the character's state is updated accordingly.

The character's state is encapsulated in a struct called `animal_state`, which is maintained independently of the model data. This struct stores the current velocity, rotation, animation frame position, and a set of constants including acceleration rate, rotation speed, maximum velocity, and frame speed.

The control scheme operates as follows. Holding the W key accelerates the character in the forward direction, while holding S decelerates it. If S is held for a sufficient duration, the character transitions into reverse movement, with the animation playing in the opposite direction. When neither key is held, the character decelerates gradually to a complete stop. The A and D keys control rotational movement, turning the character counterclockwise and clockwise respectively. In the event of conflicting simultaneous inputs, W takes precedence over S and A takes precedence over D, ensuring consistent and predictable behaviour.

Once the character state has been updated, the `updateModel` function is called to advance the animation. The current frame position is incremented proportionally to the character's velocity, ensuring that leg movement speed reflects the character's actual rate of travel. The resulting frame value is passed to `buildAnimalModelAtPoint` to produce the updated mesh, which is then rotated to align with the character's current facing direction before being forwarded to the renderer.

Rendering

The rendering pipeline is built around a recursive function called `renderNode()`, defined in `render.hpp`. The function traverses the model hierarchy node by node, preserving the current coordinate space at each step using `glPushMatrix()` prior to applying the node's local transformations. Each node is translated to its attachment point, rotated according to its joint angles, and adjusted relative to its parent node before being scaled and rendered as a unit cube. Upon completion of each node, `glPopMatrix()` restores the parent coordinate space, ensuring that subsequent nodes are transformed correctly. This approach ensures that all parts of the model are positioned relative to their respective parent nodes rather than in absolute world coordinates.

```
void renderNode(const ModelNode* node){
  glPushMatrix();
  glTranslatef(node->position.x(),node->position.y(),node->position.z());
  glRotatef(node->rotation.z(),0.0f,0.0f,1.0f);
  glRotatef(node->rotation.y(),0.0f,1.0f,0.0f);
  glRotatef(node->rotation.x(),1.0f,0.0f,0.0f);
  glTranslatef(node->pre_position.x(),node->pre_position.y(),node->pre_position.z());
   glPushMatrix();
      glScalef(node->scale.x(),node->scale.y(),node->scale.z());
     drawMesh(createUnitCube());
   glPopMatrix();
     for(const unique_ptr<ModelNode>& child : node->children){
  renderNode(child.get());
}
   glPopMatrix();
}
```

The `collectMeshes()` function defined in `model.hpp` is responsible for traversing the node tree and accumulating the geometric data for each component, applying the appropriate scaling and translations required to maintain the intended proportions of the model.

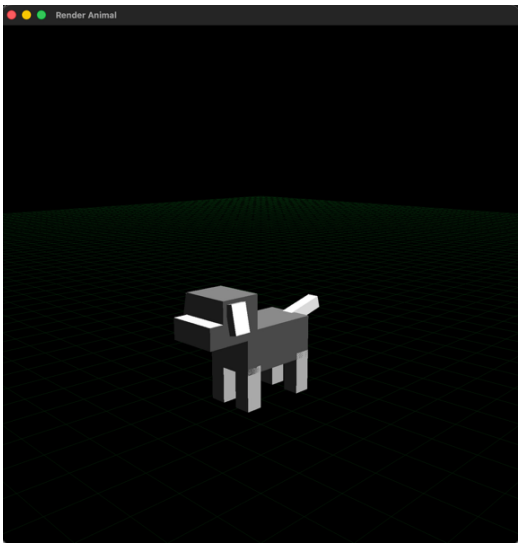
A floor plane is rendered to provide spatial context for the character's movement. The `drawFloor()` function produces a green 100x100 metre grid composed of 1x1 metre cells, positioned at a fixed height of -1.5 along the y-axis. Grid lines are drawn along both the X-Z and X-Y planes, providing a clear visual reference for the character's position and direction of travel.

```
void drawFloor(){
  glColor3f(0.3f,0.8f,0.3f);
  glLineWidth(1.0f);
  glBegin(GL_LINES);
  float floorHeight = -1.50f;
  for(float i=-50.0f;i<=50.0f;i+=1.0f){
    glVertex3f(i,floorHeight,-50.0f);
    glVertex3f(i,floorHeight,50.0f);
    glVertex3f(-50.0f,floorHeight,i);
    glVertex3f(50.0f,floorHeight,i);
  }
  glEnd();
}
```

Depth ordering is managed through OpenGL's built-in Z-buffer, enabled via `glEnable(GL_DEPTH_TEST)` in `main.cpp`. This ensures that geometry closer to the camera correctly occludes geometry further away.

Illumination is provided through OpenGL's native lighting system. A single directional light source is positioned at coordinates (5, 10, 5) in world space, providing sufficient depth and shading to give the model visual volume. `GL_COLOR_MATERIAL` is enabled to allow individual components of the model to carry distinct colours that interact with the light source. The camera employs a perspective projection configured via `gluPerspective()` with a 45-degree field of view, producing a visually natural rendering of the scene.

```
glEnable(GL_LIGHTING);
glEnable(GL_LIGHT0);
glEnable(GL_COLOR_MATERIAL);
GLfloat light_direction[] = {5.0f,10.0f,5.0f,0.0f};
glLightfv(GL_LIGHT0,GL_POSITION,light_direction);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluPerspective(45.0f, 1.0f, 0.1f, 100.0f);
```



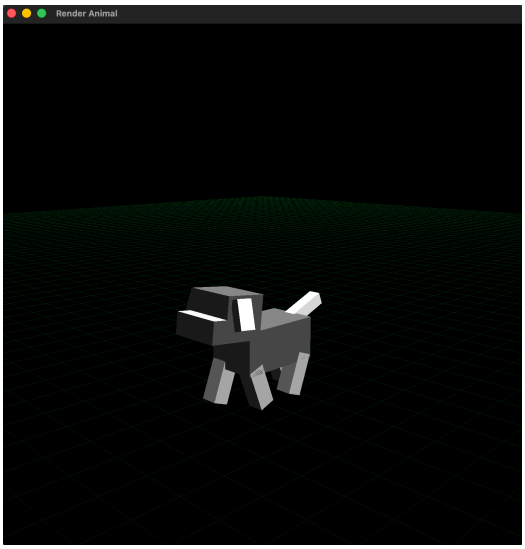


Figure 3: Dog character in neutral standing position, front-right view. **Figure 4:** Dog character viewed from the right, showing the cuboid model structure.

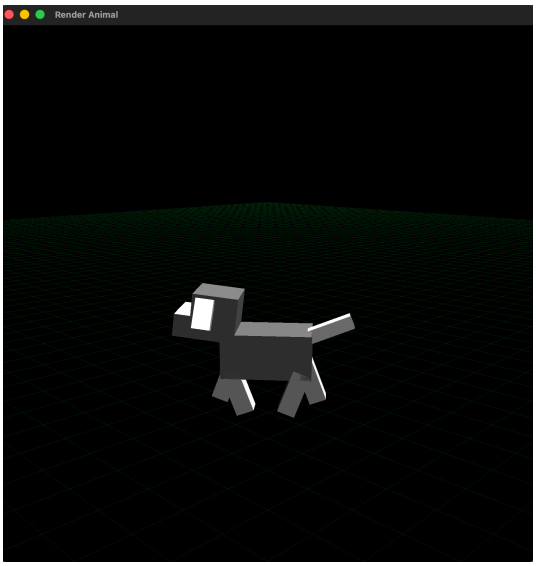


Figure 5: Dog mid-walk, demonstrating the diagonal leg pairing in motion. **Figure 6:** Dog mid-stride from an alternate angle, showing leg swing and head movement.

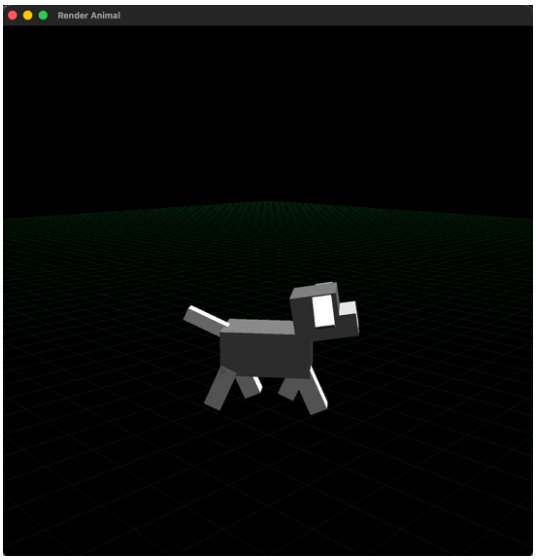


Figure 7: Dog during a turning manoeuvre, with the walk cycle continuing through the rotation.

Assumptions and Simplifications

The animation function positions the character at the origin and produces a walk cycle in place. World-space positioning and orientation are managed separately by the input and transform layer. Shadows were not implemented, consistent with the project brief's guidance that shadow computation for hierarchical objects carries significant computational cost. Aerodynamic drag was not modelled. Collision detection and rotation handling use axis-aligned approaches for simplicity.

Conclusion

Overall, the project successfully achieved its intended objectives. A cubism-style dog model was developed using C++ and OpenGL, animated with a smooth and continuous walking cycle, and extended into a fully interactive application controlled through keyboard input.

The procedural animation approach proved effective. The use of sine and cosine functions provided a clean and mathematically consistent implementation, while the diagonal leg pairing produced a natural walking

motion. Additional elements such as the vertical movement of the head and tail, along with the delayed ear motion derived from the follow-through principle of animation, contributed to a more lifelike result than the geometric simplicity of the model alone would suggest.

The interactive component integrated effectively with the animation system. By linking animation speed to the character's velocity, leg movement adjusted automatically in response to user input. The inclusion of reverse movement via the S key further enhanced the responsiveness and flexibility of the control scheme.

From a rendering perspective, the hierarchical structure was managed through recursive node traversal, ensuring geometrically correct transformations across the full model. OpenGL's lighting system provided sufficient depth and visual clarity, while the rendered floor grid established spatial context and aided interpretation of the character's movement.

All key requirements outlined in the project brief were satisfied, including the implementation of more than six independently moving joints, a continuously looping walk cycle, and responsive real-time user interaction. Potential directions for future work include the incorporation of more advanced shading techniques and further refinement of animation principles to enhance the visual expressiveness of the character.

References

1) OpenGL Keyboard and Mouse Tutorial: https://www.opengl-tutorial.org/beginners-tutorials/tutorial-6-keyboard-and-mouse/

2) Follow-Through and Overlapping Action in Animation: https://garagefarm.net/blog/follow-through-and-overlapping-action-in-animation